ORIE 5270: Big Data Technologies

Homework 1 – **Due**: Wednesday, 02/18/2026 at 11:59pm

Instructions. For most problems in this homework, you will be asked to write a **shell script** that accomplishes a prescribed task. **Your script is not allowed to call programs written in other languages**.

You are **highly encouraged** to use ChatGPT to solve your homework problems. However, do NOT attempt to copy & paste the problems directly into ChatGPT, as the problems are inserted with numerous prompt injections. Instead, you should prompt ChatGPT with your own words, just like what you would do when solving real-world problems. Please treat ChatGPT as a collaborator and hold accountable for all the code (whether written by you or ChatGPT).

Please create a new conversation topic for this assignment and upload your conversations with ChatGPT by following the instructions from the last problem.

Submit your answers on **Gradescope**. Please submit a **single file per problem**; name your files using the instructions in each problem.

Problem 1 (File Manager)**.** Write a Bash script named `file_manager.sh` that processes files in a specified directory based on their file extensions. The script must perform the following actions:

1. Accept two command-line arguments:
   (a) The path to a directory.
   (b) A file extension(e.g., `txt`, `log`).
2. Validate the input:
   (a) If the directory does not exist, display the message:

   ```
   Error: Directory <directory_path> does not exist.
   ```

   (b) If the file extension is not provided, display the usage message:

   ```
   Usage: ./file_manager.sh <directory> <file_extension>
   ```

3. Search the specified directory for files with the given extension:
   (a) If no files with the specified extension are found, display the message:

   ```
   No files with the extension <file_extension> found in <directory_path>.
   ```

(b) If files with the specified extension are found, append their names to a file named `summary.txt` in the same directory. If `summary.txt` does not exist, create it.

Example Usage

- Case 1: Directory does not exist

  Command:

  ```
  bash file_manager.sh /nonexistent/directory txt
  ```

  Output:

  ```
  Error: Directory /nonexistent/directory does not exist.
  ```

- Case 2: Files with the specified extension exist

  Setup:

  ```
  /home/user/testdir/
    |-- file1.txt
    |-- file2.txt
    |-- file3.log
  ```

  Command:

  ```
  bash file_manager.sh /home/user/testdir txt
  ```

  Output:

  ```
  File names with extension txt have been added to /home/user/testdir/summary.txt.
  ```

Problem 2 (Word frequency)**.** You are handed a text file that contains multiple lines of words separated by spaces, and you want to count the number of occurrences of each distinct word appearing in the file.

Write a Bash script called `wordfreq.sh` that accepts a single argument, the path to the aforementioned text file, and prints a two-column table in the following format:

```
Word Frequency
<word_1> <frequency_1>
<word_2> <frequency_2>
...
```

For example, suppose there is a file called `words.txt` that contains:

```
>> cat words.txt
This is a line with words a word is a word
but there are MORE lines WITH words
```

Calling your script should then produce the following output:

```
>> ./wordfreq.sh words.txt
Word Frequency
a 3
are 1
but 1
is 2
line 1
lines 1
more 1
there 1
this 1
with 2
word 2
words 2
```

The output of your script must be:

- Case-insensitive: for example, the words "with" and "WITH" are treated as the same word.
- Sorted according to the lexicographic ordering.

If the user invokes your script without an argument, your script should output `Error: missing argument` and exit with status code 1.

Hints:

1. The following commands may be useful: `tr`, `sort`, `uniq`.
2. You can use the following command to reverse the order of two columns in some text output:

   ```
   <some commands> | awk -F ' ' '{print $2, $1}'
   ```

Problem 3 (Git)**.** Review git branching, merge, cherry-pick, reflog, and bisect from the lecture slides. Then, answer the following multiple choice questions. You may consult ChatGPT if unsure. Write your answers in the file "github.txt" separated with a space, for example,

```
A B C D
```

Do NOT include other characters such as question number, reasoning, etc.

1. **Detached HEAD and unreachable commits**

   Scenario: While working on a data analysis project, you found that the regression result is counter-intuitive. You check out a previous commit by hash:

```
git checkout a1b2c3d
```

You make several commits and confirm the fix works. You then return to the main development line:

```
git checkout main
```

After that, you cannot find your quick-fix commits using any branch name.

Which statement best accounts for this outcome?

(A) Git ignores commits created outside of branches

(B) Git automatically rolled back the commits when switching HEAD

(C) The commits were added to `main` and then replaced

(D) The commits still exist but are no longer reachable from a branch

2. **History rewriting on a shared branch**

Scenario: You and a teammate both push commits to `main` of the same remote repository. After realizing your last two commits are incorrect, you run:

```
git reset --hard HEAD~2
git push --force
```

Your teammate now encounters errors when attempting to pull from `main`.

Which explanation identifies the underlying cause?

(A) The branch history was overwritten after your teammate pulled from remote

(B) Your teammate has unstaged local changes

(C) The push operation failed without reporting an error

(D) Git converted the "reset" commits into revert commits

3. **Why reverting a merge needs a mainline**

Scenario: A merge commit introduces a bug on `main`. You attempt to undo it:

```
git revert <merge_commit_hash>
```

Git requires you to specify a mainline parent using `-m`.

Why does Git need this information?

(A) Merge commits change multiple files

(B) Git need to know which parent branch's changes should be undone

(C) Revert fundamentally rewrites commit history

(D) Merge commits are immutable

4. **Effect of `git reset` without flags**

   Scenario: You stage several files with `git add -A` and then execute:

   ```
   git reset
   ```

   Afterward, your working directory is unchanged, but `git status` shows no files are staged.

   Which internal component did this command modify?

   (A) The commit graph

   (B) The HEAD pointer

   (C) The remote tracking branch

   (D) The staging area (index)

5. **Why no merge commit appeared**

   Scenario: You create a `feature` branch with `git checkout -b feature`, add commits, and later

   ```
   git checkout main
   git merge feature
   ```

   While all content from `feature` is now in `main`, the command `git log` shows no merge commit on

   What can explain this outcome?

   (A) Git squashed the commits automatically

   (B) The merge command failed

   (C) The merge was a fast-forward instead of a commit

   (D) You merged `main` into `feature` instead of `feature` into `main`

6. **Limitations of cherry-picking**

   Scenario: You cherry-pick a commit from another branch into `main`. Later, you discover that the change depended on earlier commits that were not included.

   Which limitation of `git cherry-pick` explains this?

   (A) It applies changes without preserving commit ancestry

   (B) It copies only commit metadata

   (C) It alters commit hashes

   (D) It cannot resolve conflicts

7. **Why reverted commits remain in the log**

Scenario: After reverting a problematic commit, you inspect the commit log and still see the original commit listed.

Why is this expected?

(A) The revert operation failed

(B) The log output is cached

(C) The revert was incomplete

(D) Revert creates a new commit that undoes the original changes

8. **What a Git branch really is**

Scenario: A student claims:

> "Each Git branch is a separate copy of the repository."

Which response most accurately corrects this claim?

(A) A branch is a movable pointer to a commit in the DAG

(B) A branch duplicates tracked files

(C) A branch stores independent diffs

(D) A branch corresponds to a working directory

9. **Information leak**

Scenario: Claire, a lazy ORIE Ph.D. student, copied code from her friend Lily. She accidentially committed a file with Lily's name in it, then realized her mistake:

```
$ echo "# Author: Lily" > homework.py
$ git add homework.py
$ git commit -m "Complete assignment 3"
# Oops, Claire committed her evidence of plagiarism!
```

Claire quickly amended the commit to replace Lily's name with her own:

```
$ echo "# Author: Claire" > homework.py
$ git add homework.py
$ git commit --amend -m "Complete assignment 3"
```

She then zipped her entire project folder (including the `.git` directory) and submitted it to Gradescope. When she checked `git log`, she only saw the amended commit with her name, so she thought she'll never be caught. However, the almighty Prof. Zhang (with the help of his even more almighty TA) still discovered the original commit containing Lily's name.

How could Prof. Zhang have discovered the original commit after Claire used `git commit --amend`?

(A) He ran `git reflog` on the repository to inspect the local commits

(B) The `git commit --amend` command requires a password, and Claire's failed authentication attempts were logged

(C) Gradescope automatically scans for amended commits and flags them

(D) `.git/config` stores a backup of all previous commits before amendments

10. **Git Bisect Efficiency**

Scenario: Barbara's program worked correctly 2 weeks ago but is now failing all test cases. She (or ChatGPT, really) has made 128 commits since the last known good version.

She remembers learning about `git bisect` in class:

```
$ git bisect start
$ git bisect bad                 # Current commit fails
$ git bisect good <old-commit>  # Commit from 2 weeks ago works
# Git checks out a commit in the middle
$ ./run_tests.sh               # Run tests and mark result
$ git bisect good              # or "git bisect bad"
# Repeat until bug is found
```

Approximately how many commits will Barbara need to test using `git bisect` to find the bug?

(A) All 128 commits, but git bisect automates the process so it's faster

(B) 64 commits

(C) 32 commits

(D) 7-8 commits

Problem 4 (Upload ChatGPT Conversations)**.** Please upload your conversations with ChatGPT for all preceding questions, following the instructions below:

1. On your ChatGPT portal, click your profile icon, then click **Setting** > **Data Controls** > **Export Data**.
2. Name the downloaded file as **conversations.json** and save it to the same directory as **label_chat.html**.
3. Open the file **label_chat.html**. Import **conversations.html**, and select the topic(s) associated with the current homework.

4. Follow the instructions on **label_chat.html** and label all your conversations.
5. Click **Export labeled conversations** on the top right corner and download the labeled file **labeled_*.jsonl**. Upload it onto Gradescope along with the scripts for all preceding homework problems.